

• Section - A (2x5=10)

1. What is intermediate code?

Ans Intermediate code is a machine-independent representation of a program, generated between high-level source code and machine code by the compiler. It simplifies optimization and code generation for multiple architectures.

eg: Three-Address Code (TAC)

$$t1 = a + b$$

$$t2 = t1 * c$$

2. What you mean by dead code optimization?

Ans Dead code optimization is a compiler technique that removes unnecessary or unreachable code to improve efficiency. It reduces memory usage and execution time.

eg:- `int x = 5;`

`x = 10; // 'x = 5' is dead code since its value is never used.`

3. What is back patching?

Ans Backpatching is a method to handle forward jumps in code generation. It delays address resolution of jump statement and later updates them when the target is known. It is used in Boolean expressions, loops and conditional statements.

4. Explain in brief about control stack.

Ans A control stack is a data structure used by a compiler to manage function calls, recursion, and scope handling during program execution. It follows the LIFO (Last-In-First-Out) principle, where each function call pushes an activation record onto the stack and when the function completes, it is popped off.

5. Write down the benefits of using an intermediate code generation over direct code generation.

Ans Benefits of Using Intermediate Code over Direct code generation:

- **Machine Independence** - Can be translated into multiple target architectures.
- **Optimization** - Allows compiler optimizations before final machine code generation.
- **Simplifies Code Generation** - Reduces complexity by breaking translation into stages.
- **Portability** - Same intermediate code can be used for different machines.

• Section-B (7x3=21)

1. Define runtime environment. What are the issues in runtime environment?

Ans The runtime environment refers to the collection of resources and mechanism that a program uses while executing. It includes the memory management system, execution model, libraries, and the system stack that handles function calls, variable storage and resource allocation during program execution. The runtime environment supports program execution by providing essential services like input/output operations, memory allocation and exception handling.

• Issues in Runtime Environment -

- (i) **Compatibility** - Programs may not run correctly if the runtime environment lacks required libraries, dependencies or specific versions.
- (ii) **Performance** - Resource limitations (e.g. memory, CPU) or inefficient resource management can slow down execution.
- (iii) **Security** - Vulnerabilities in the runtime environment can expose the program to attacks, such as unauthorized access or data breaches.
- (iv) **Portability** - Programs may behave differently across environments due to variations in OS, hardware or configurations.
- (v) **Error Handling** - Runtime errors (e.g. null pointers, memory leaks) can crash the program if not properly managed.

2. Explain the Peephole Optimization in detail.

Ans Peephole Optimization is a Local code optimization technique where the compiler examines a small set of instructions (a "peephole") in the generated code and replaces them with more efficient sequences. It focuses on improving performance, reducing code size, and eliminating redundant operations.

• Key techniques in peephole optimization -

(i) Redundant Instruction Elimination - Removes unnecessary instructions such as loading a value into a register when it is already present.

ex: - `MOV R1, A`
`MOV A, R1` → (Unnecessary move is removed)

(ii) Constant Folding - Replaces constant expressions with their computed value at compile time.

ex: `ADD R1, 2, 3` → `MOV R1, 5`

(iii) Strength Reduction - Replaces costly operations with more efficient ones.

ex: `MUL R1, 2` → `SHL R1, 1` (Multiplication replaced with shift operation).

(iv) Eliminating Unreachable Code - Removes code that will never be executed.

ex:

(v) Algebraic Simplification - Simplifies expression to reduce computation.

ex: `MUL R1, 1` → `MOV R1, R1` (Multiplication by 1 is unnecessary).

• Benefits:

- Improve execution speed.
- Reduces code size.
- Enhances resource utilization.

Q3. Explain the DAG representation of Flow graph with suitable example.

Ans. A Directed Acyclic Graph (DAG) is used in compiler optimization to represent expression and basic blocks in a flow graph efficiently. It helps in eliminating common subexpressions, reducing redundant calculations and optimizing code generation.

ex:- Consider following basic block:-

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$t_3 = a + b$$

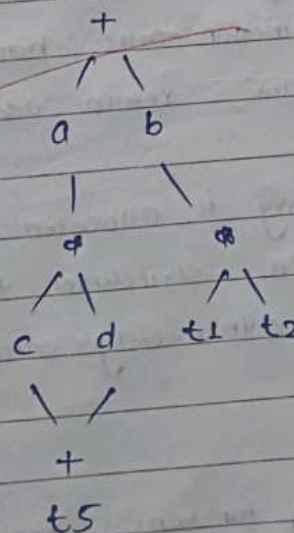
$$t_4 = t_3 * d$$

$$t_5 = t_2 + t_4$$

* DAG Representation :-

- (i) Creates node for operands (a, b, c, d).
- (ii) Identify common subexpressions (a+b appears twice).
- (iii) Reduce redundancy by using the same code node for repeated expressions.

DAG structure :-



Optimization from DAG: -

- $t_3 = a+b$ is redundant and replaced with t_1 .
- The optimized code becomes:

$$t_1 = a+b$$

$$t_2 = t_1 * c$$

$$t_4 = t_1 * d$$

$$t_5 = t_2 + t_4$$

- Section-C (11+3=33)

1.(A) Explain the storage allocation techniques in detail.

Ans Storage Allocation Techniques are methods used by compilers to allocate memory for variables, data and program instructions during execution. The choice of technique depends on the scope, lifetime and size of the data.

The Three main techniques are:-

- Static Allocation** - Memory is allocated at compile time and cannot be changed during execution.
 - Used for: Global variables, static variables and constants.
- Stack Allocation** - Memory is allocated and deallocated dynamically using a stack for function calls, handling local variables, function parameters and return addresses.
- Heap Allocation** - Memory is allocated dynamically at runtime for objects and data structures that need flexible memory management such as dynamically allocated arrays and linked lists.
 - How it works -
 - Memory is allocated using functions like `malloc()` or `new`.
 - Memory is explicitly deallocated using `free()` or `delete`.

Other's allocation techniques are-

(iv) Automatic (Local) Allocation - memory is allocated automatically for local variables when a block or function is entered and deallocated when the block or fun. exits.

Used for - local variables within a fun. or block.

(v) Dynamic allocation -

memory is allocated at runtime based on program requirements.

Used for - D.S. like arrays, linkedlist, and objects.

(B) Translate the following statement, which alters the flow of control of expressions and generate the three-address code for it.

if $(a < b)$ $\&\&$ $(c < d)$ then t_1 else t_2 .

Ans. Given Statement -

if $(a < b)$ $\&\&$ $(c < d)$ then t_1 else t_2 ;

Step 1: Convert to Boolean Expressions.

Logical AND ($\&\&$) requires evaluation of both conditions $(a < b)$ and $(c < d)$.

Step 2: Generation of Three-Address Code

L1: if $a < b$ goto L2
goto L4

L2: if $c < d$ goto L3
goto L4

L3: $t = t_1$
goto L5

L4: $t = t_2$

L5:

Explanation: • if $a < b$ is false, jump to L4 (execute 'else' part)
• If $a < b$ is true, check $c < d$.
• If both condⁿ hold, execute t_1 , otherwise execute t_2 .

Q: A. Explain the type of Three = Address Code with examples.

Ans: Three = Address Code (TAC) is an intermediate representation used in compilers, where each instruction has at most three operands (variables or constants) and performs a single operation. The main type of TAC are -

(i) Assignment Statements - Assigns value to variables.

Syntax: $x = y$ or x

ex: $a = b + c;$

TAC: $t1 = b + c$

$a = t1$

(ii) Arithmetic Operations - Uses temporary variables for calculations.

ex: $t1 = b + c$

$t2 = a + t1$

(iii) Conditional Jumps - Represents a jump based on a condition.

Syntax: "If x rel op y goto L "

ex: If $a > b$ goto $L1$

(iv) Unconditional jumps - Represents a jump to a specific label (used for control flow).

Syntax: goto L

ex: goto label

(v) Indexed Assignments - Represents array element access or assignment.

Syntax: $x = y[i]$ or $y[i] = x$

ex: $t1 = arr[2]$

$a = t1$

$arr[3] = b$

(vi) Function Calls and Return - Representing calling a function and storing the result.

Syntax: $x = call f$

ex: Param x

TAC: Param y

$t1 = call func$

$x = t1$

} - $x = func(x, y);$

- (2) (B) Translate the following statement, which alters the flow of control of expressions and generate the TAC for it -

for ($i=1; i \leq n; i++$)

{

$x = a + b * c$

}

Ans Given statement:- for ($i=1; i \leq n; i++$) {
 $x = a + b * c$ }

Three - Address code (TAC):-

- (i) Initialization:- Initialize i to 1.

$i = 1$

- (ii) Condition Check:- check if $i \leq n$, if true, Proceed to the loop body otherwise exit the loop.

L1: if $i \leq n$ goto L2
goto L3

- (iii) Loop Body:- Compute $b * c$ and store in a temp var $t1$.
compute $a + t1$ and store result in x .

L2: $t1 = b * c$
 $x = a + t1$

- (iv) Increment:- increment i by 1.

$i = i + 1$

- (v) Repeat:- Jump back to the condⁿ check.

goto L1

- (vi) Exit:- Label for exiting the loop. L3:

Final Three - Address code -

$i = 1$

L1: if $i \leq n$ goto L2
goto L3

L2: $t1 = b * c$
 $x = a + t1$
 $i = i + 1$
goto L1

L3:

Q3. (A) Explain quadruple with the help of a suitable example.

Ans. Quadruples are a type of intermediate representation used in compilers to represent three-address code in a structured form. Each quadruple consists of four fields.

- (i) Operator - Specifies the operation (eg: +, -, *, /, =, if).
- (ii) Argument 1 - First Operand
- (iii) Argument 2 - Second Operand
- (iv) Result - Stores the result of the operation.

Ex:-

$z = a + b * c;$

Step-by-step Quadruples:-

(i) Multiply b and c:-

- Operator: *
- operand 1: b
- operand 2: c
- Result: t1 (temp var)

(*) (, b, c, t1)

(ii) Add a and t1:-

- Operator: +
- operand 1: a
- operand 2: t1
- Result: t2 (temp var)

(+, a, t1, t2)

(iii) Assign t2 to z:-

- Operator: =
- operand 1: t2
- operand 2: (empty, an assignment is a unary operation)
- Result: z

(=, t2, -, z)

(3) (8)

• Final quadruples Table:-

Operator	Operand 1	Operand 2	Result
∞	b	c	t1
+	a	t1	t2
=	t2	-	X

(3) (B) Translate the following statement, which alters the flow of control of expressions and generate the three-address code for it.

Switch (i)
{

Case 1 :

$$x1 = a1 + b1 * c1$$

Break ;

Case 2 : $x2 = a2 + b2 * c2$

Break ;

Default :

$$x3 = a3 + b3 * c3$$

Break ;

}

Ans Three-Address code for Given Statement:-

(i) Check for Case 1 :- compare i with 1. ~~if i == 1~~
• if $i == 1$, go to L1.

(ii) Check for Case 2 :- if $i == 2$, go to L2.

TAC:- if $i == 2$ goto L2

(iii) Default Case :- if no case matches, go to L3.

TAC:- goto L3

(iv) Case 1 Block:-

• compute $x1 = a1 + b1 * c1$

• go to the end (L4).

TAC:- L1: $t1 = b1 * c1$

$x1 = a1 + t1$

goto L4

(v) Case 2 Block:-

• Compute $x2 = a2 + b2 * c2$

• Go to the end (L4).

TAC: $t2 = b2 * c2$

$x2 = a2 + t2$

goto L4

(vi) Default Block:-

• Compute $x3 = a3 + b3 * c3$

TAC:- $t3 = b3 * c3$

$x3 = a3 + t3$

(vii) End of switch - Marks the end of the switch.

L4:

• Final Simplified TAC:-

if $i == 1$ goto L1

if $i == 2$ goto L2

goto L3

L1: $t1 = b1 * c1$

$x1 = a1 + t1$

goto L4

L2: $t2 = b2 * c2$

$x2 = a2 + t2$

goto L4

L3: $t3 = b3 * c3$

$x3 = a3 + t3$

L4;

the end.